

1

Rephrase
title
to be
more
im-
pact-
ful

Fourier Neural Operators using Wavelet Encodings for Solving Eigenvalue PDEs To Simulate Acoustic Metamaterial Displacement Fields

Han Zhang^a, Cynthia Rudin^b, Johann Guilleminot^a, L. Catherine Brinson^a

^aDepartment of Mechanical Engineering and Materials Science, Duke University, Durham, NC, USA

^bDepartment of Electrical and Computer Engineering, Duke University, Durham, NC, USA

Abstract

Neural operator models have in recent years demonstrated impressive capabilities in solving linear and non-linear non-eigenvalue PDEs, but the ability for these operator models to solve eigenvalue PDEs, where the PDE constants must simultaneously be solved along with a solution field, has not yet been demonstrated. In this paper, we illustrate the effectiveness of Fourier Neural Operators (FNOs) combined with wavelet-based encoding techniques for PDE parameters to simultaneously and inexpensively solve multiple eigenmodes of the wave equation for elastic solids, corresponding to multiple deformation modes from acoustic waves propagating through arbitrary metamaterial geometries. In this paper a novel wavelet encoding schema for PDE constants is shown, which allows the FNO model to be used to provide multiple valid solutions to an eigenvalue PDE, as opposed to the typical single solution of previous works. This paper also demonstrates and explains why a purely spatial or reciprocal encoding of the same parameters will not result in an FNO model with the same eigenvalue PDE solving capability.

Keywords: Neural Operator, Metamaterials, Acoustics, Wavelet Transform, Machine Learning

Table of Contents

25	Table of Contents	2
26	1 Introduction	4
27	2 Methodology	6
28	2.1 Fourier Neural Operator	6
29	2.2 Metamaterial Design Space	9
30	2.3 Wave Propagation Simulations	11
31	2.4 Input Wavelet Encoding	12
32	2.5 Dataset Construction	14
33	2.6 Training Procedure	15
34	3 Results and Discussion	17
35	3.1 Displacement Field Prediction	17
36	3.1.1 Continuous Geometries	18
37	3.1.2 Discrete Geometries	21
38	3.1.3 Continuous Versus Discrete Performance	24
39	3.2 Dispersion Band Reconstruction	25
40	3.3 Dependence of Prediction Error on Band and Boundary Length . . .	27
41	3.4 Band and Wavevector Encoding	29
42	3.5 Output Datatype	30
43	4 Conclusions	31
44	S1 Supplementary Information	33
45	S1.1 Model and Training Hyperparameters	33
46	S1.2 Loss Criterion	33
47	S1.3 Output Wavelet Encoding	38
48	S1.4 Algorithms for Gabor Wavelet Encoding	39

49 1. Introduction

50 The numerical solution of partial differential equations (PDEs) underpins modern
51 scientific computing and engineering design. Applications ranging from structural
52 mechanics and fluid dynamics to wave propagation and materials engineering rou-
53 tinely rely on computationally intensive numerical methods such as finite element
54 analysis (FEA) to obtain accurate solutions. While these methods are highly reliable,
55 their computational cost can become prohibitive in settings that require repeated
56 evaluations, such as optimization, uncertainty quantification, inverse design, and
57 real-time control.

Citation
needed

58 Recent advances in machine learning have introduced neural operators as a promis-
59 ing alternative to traditional numerical solvers. Unlike conventional neural networks
60 that learn mappings between finite-dimensional vectors, neural operators learn map-
61 pings between function spaces, allowing them to approximate entire solution operators
62 for families of PDEs. Models such as the Fourier Neural Operator (FNO) have demon-
63 strated remarkable accuracy and computational efficiency across a variety of linear
64 and nonlinear PDE problems while maintaining strong generalization capabilities
65 across discretizations and geometries. These developments have generated significant
66 interest in using neural operators as surrogate models for computational physics
67 applications.

Citation
needed

68 Despite this progress, existing neural operator architectures have primarily been
69 developed and evaluated on PDEs whose governing parameters are known a priori
70 and whose solutions are uniquely determined by the provided inputs. A large class
71 of physically important problems, however, are governed by eigenvalue PDEs, in
72 which both the solution field and one or more unknown PDE parameters must be
73 determined simultaneously. Examples include vibration analysis, quantum mechanics,
74 photonics, acoustics, and wave propagation in metamaterials. In these problems,
75 multiple valid solutions may exist for the same physical domain, corresponding to
76 different eigenvalue-eigenvector pairs. The ability to represent and distinguish among
77 these multiple solutions is essential for accurately modeling the underlying physics.

Citation
needed

Citation
needed

78 Acoustic metamaterials provide a particularly compelling example of this chal-

79 lenge. These architected materials derive their properties from geometric structure
 80 rather than composition alone, enabling engineered interactions with propagating
 81 waves. Understanding and designing such systems often requires computing multiple
 82 deformation modes associated with different resonant frequencies and wave behaviors.
 83 Traditionally, these modes are obtained through computationally expensive eigenvalue
 84 analysis using finite element methods. A neural operator capable of accurately predict-
 85 ing multiple deformation modes directly from material geometry would significantly
 86 accelerate such iterative design and analysis processes.

Citation
needed

Citation
needed

87 However, extending neural operators to eigenvalue PDEs is nontrivial. Standard
 88 operator-learning formulations implicitly assume a one-to-one mapping between in-
 89 puts and outputs, whereas eigenvalue problems admit multiple valid solutions for a
 90 given geometry. As a result, existing FNO architectures lack a mechanism for deter-
 91 ministically selecting among different eigenmodes. Furthermore, the representation of
 92 eigenvalue-related information must be compatible with the spectral architecture of
 93 the FNO, which processes information jointly in both spatial and frequency domains.

94 In this work, we introduce a framework that enables Fourier Neural Operators to
 95 solve eigenvalue PDEs associated with elastic wave propagation in acoustic metama-
 96 terials. The key idea is to augment the model inputs with wavelet-based encodings
 97 of PDE parameters that uniquely identify individual eigenmodes while remaining
 98 compatible with the FNO architecture. We demonstrate that these encodings allow
 99 a single neural operator to learn multiple valid solutions corresponding to distinct
 100 deformation modes and to reproduce those modes deterministically on demand. We
 101 further show that conventional spatial encodings and purely spectral encodings do
 102 not provide the same capability, highlighting the importance of the proposed wavelet
 103 representation.

104 The contributions of this paper are threefold:

- 105 1. We demonstrate, to the best of our knowledge, the first application of Fourier
 106 Neural Operators to the solution of eigenvalue PDEs involving unknown modal
 107 parameters.
- 108 2. We introduce a wavelet-based encoding strategy that enables deterministic

Citation
 needed,
 rephrase
 to be
 sharp
 and
 up to
 date,
 indi-
 cate

109 selection among multiple eigenmodes within a single neural operator framework.

110 3. We investigate the interaction between input representations and the dual spatial-
111 spectral structure of the FNO, providing insight into why wavelet encodings
112 are effective whereas purely spatial or purely reciprocal representations are not.

113 Together, these results extend the applicability of neural operators beyond con-
114 ventional forward PDE problems and establish a pathway toward efficient surrogate
115 modeling of eigenvalue-driven physical systems, including acoustic metamaterials and
116 other wave-based engineering applications.

117 2. Methodology

118 This body of work involves several different methods for generating and then
119 training with data to enable FNO models to simulate acoustic metamaterials, a
120 practical and nontrivial eigenvalue PDE problem. Each method is detailed in separate
121 subsections, starting with the materials simulation side of the procedure, and then
122 moving into the machine learning aspects. The reader is encouraged to read or skip
123 any of these subsections depending on their interest in technical details.

124 2.1. Fourier Neural Operator

125 The principal theoretical motivation for Fourier Neural Operators is their universal
126 approximation property over function spaces. Let $\mathcal{A}$ and $\mathcal{U}$ denote Banach spaces of
127 input and output functions, respectively, and let

$$\mathcal{G} : \mathcal{A} \rightarrow \mathcal{U} \tag{1}$$

128 be a continuous nonlinear operator. It has been shown that, given sufficient latent
129 width and Fourier modes, an FNO can approximate $\mathcal{G}$ arbitrarily well on compact sub-
130 sets of $\mathcal{A}$. Unlike conventional neural networks, which approximate finite-dimensional
131 functions, FNOs approximate operators between infinite-dimensional function spaces
132 by learning a sequence of global integral operators parameterized in the Fourier
133 domain.

citation

134 This theoretical framework assumes that both the input and output are continuous
 135 fields of infinite resolution. In practice, however, these fields are sampled on a finite
 136 computational grid. The continuous Fourier transform is therefore replaced by
 137 a discrete Fourier transform (DFT), and only a finite number of Fourier modes
 138 are retained during training. Consequently, the learned model approximates the
 139 continuous solution operator through its discrete representation while preserving the
 140 same spectral operator-learning architecture.

141 For the present work, the operator acts on discretized fields over a 32×32 pixel
 142 spatial grid. The input field consists of three channels containing encoded information
 143 for the metamaterial geometry, the stimulatory wavevectors, and the band number.
 144 The output field consists of five channels corresponding to the predicted eigenvalue, or
 145 frequency, and eigenvector, or displacement field, of the resulting deformation mode.
 146 Thus, the discretized learned operator takes the form

$$\mathcal{G}_\theta : \mathbb{R}^{32 \times 32 \times 3} \rightarrow \mathbb{R}^{32 \times 32 \times 5}. \quad (2)$$

147 The discretized FNO follows the standard lift–transform–project architecture:

- 148 1. **Lifting:** The input field is embedded into a higher-dimensional latent space
 149 through a pointwise lifting operator,

$$v_1(x) = \ell(a(x)), \quad \ell : \mathbb{R}^3 \rightarrow \mathbb{R}^{d_L}. \quad (3)$$

- 150 2. **Fourier layers:** A sequence of Fourier layers is applied to the latent represen-
 151 tation. Each layer updates the hidden field according to

$$v_{j+1}(x) = \sigma \left(\mathcal{F}^{-1} [R_j(k) \mathcal{F}[v_j](k)] + W_j v_j(x) + b_j \right), \quad (4)$$

152 where $\mathcal{F}$ denotes the discrete Fourier transform, $R_j(k)$ is the learnable spectral
 153 kernel, W_j is a learnable pointwise linear transformation, b_j is a learnable bias
 154 term, and σ is the activation function. GELU was used in this work because it
 155 is smooth and differentiable.

156 **3. Projection:** The final latent representation is projected back to the desired
 157 output dimension through a pointwise projection operator,

$$u(x) = p(v_n(x)), \quad p : \mathbb{R}^{d_L} \rightarrow \mathbb{R}^5. \quad (5)$$

158 FNOs are well suited for acoustic metamaterial simulation because the underlying
 159 physics is governed by spatially distributed wave equations, where the response
 160 at one location depends on the global geometry of the unit cell and the imposed
 161 wavevector. The Fourier layers are designed to use spectral convolutions to efficiently
 162 capture long-range interactions and periodic spatial structure, while the pointwise
 163 transformations preserve local geometric information. Although FNOs are commonly
 164 used as surrogate models for fixed PDE solution operators, the same operator-learning
 165 framework can be extended to eigenvalue problems by conditioning the input on
 166 modal parameters, such as the wavevector and band index, and training the network
 167 to output both the associated eigenvalue and eigenvector field. In this formulation,
 168 the FNO learns a parameterized eigensolution operator mapping geometry and modal
 169 encodings to the frequency and deformation mode of the acoustic unit cell.

170 This process is illustrated graphically in Figure 1. For our experiments, the
 171 NeuralOp Python package was used, which provides FNO model definitions with
 172 customizable numbers of layers, hidden channels, and input and output tensor sizes.
 173 The hyperparameter combinations explored, as well as the optimized final FNO
 174 configuration, are shown in Table S1 in Section 2.6. These hyperparameters are used
 175 alongside the fixed parameters of the problem space: model height 32, model width
 176 32, input channels 3, and output channels 5.

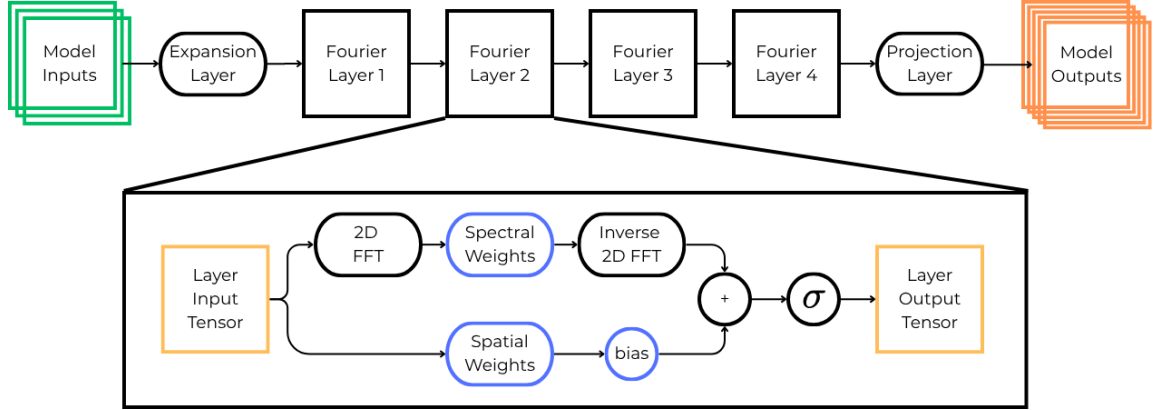


Figure 1: The FNO model architecture used in this project. The input data is first expanded according to the hidden-channels parameter, allowing multiple latent features to be represented between Fourier layers. Each Fourier layer is shown in detail in the expanded view. The data passing through a Fourier layer is processed through two learned branches: a pointwise branch, which applies a learned linear transformation in the spatial representation, and a spectral branch, which applies a fast Fourier transform (FFT), multiplies the retained Fourier modes by learned spectral weights, and then applies an inverse FFT. The outputs of both branches, along with a learnable bias term, are summed before passing through a nonlinear activation function. After passing through four Fourier layers, the latent representation is projected from the hidden-channel dimensionality to the final output dimensions.

2.2. Metamaterial Design Space

To train a surrogate model capable of predicting acoustic eigenmodes, a representative design space of acoustic metamaterials must first be defined. The design space was selected to provide sufficient geometric and material complexity to demonstrate operator learning for eigenvalue PDEs while maintaining computationally tractable finite element simulations for dataset generation.

The dataset is restricted to two-dimensional acoustic metamaterials composed of infinitely repeating square unit cells exhibiting eight-fold symmetry. Each unit cell is discretized onto a 32×32 spatial grid and represented by three material-property channels corresponding to the normalized elastic modulus, density, and Poisson’s ratio. Material properties are normalized to the interval $[0, 1]$ to improve numerical conditioning during training, while the corresponding physical values are recovered using the fixed material constants listed in Table 1. The two constituent materials were chosen to represent a stiff steel alloy and a compliant polymer.

Parameter	E_{polymer}	E_{steel}	ρ_{polymer}	ρ_{steel}	ν_{polymer}	ν_{steel}
Units	Pa	Pa	kg/m ³	kg/m ³	—	—
Value	1×10^6	2×10^9	1200	8000	0.45	0.3

Table 1: Ranges for material parameters. The hard material is chosen to be a common steel alloy and the soft material a rubbery polymer.

Unit-cell geometries are synthesized by sampling a Gaussian process with a periodic covariance kernel and subsequently enforcing eight-fold symmetry. This procedure produces smoothly varying spatial material distributions while ensuring compatibility with periodically tiled square lattices. The sampled field assumes values between 0 and 1, where 0 represents pure polymer, 1 represents pure steel, and intermediate values denote an effective mixture of the two materials below the spatial resolution of the discretization.

To investigate the influence of geometry representation on model performance, two classes of unit cells were generated. The first consists of the continuous-valued Gaussian process realizations described above and is referred to as the *continuous geometry* dataset. The second is obtained by thresholding the Gaussian process samples to binary numbers, producing geometries composed exclusively of polymer and steel. This second collection is referred to as the *discrete geometry* dataset. Each representation comprises approximately one-half of the total dataset.

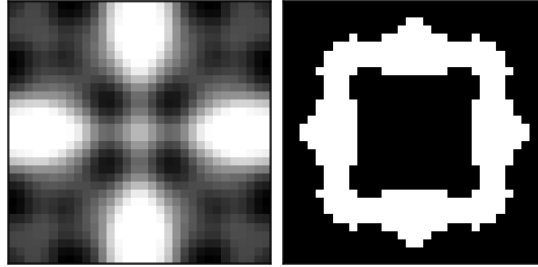


Figure 2: Example of a continuous geometry (left) and a discrete geometry (right) generated using the above process and parameters.

205 2.3. Wave Propagation Simulations

206 For each metamaterial geometry, wave propagation is simulated by solving the
 207 frequency-domain linear elastodynamic equation with finite element analysis (FEA).
 208 After spatial discretization and enforcement of Bloch periodicity, the governing
 209 problem takes the generalized eigenvalue form

$$(\mathbf{K}_r(\mathbf{k}) - \omega^2 \mathbf{M}_r(\mathbf{k})) \mathbf{u} = \mathbf{0}, \quad (6)$$

210 where ω is the angular eigenfrequency, $\mathbf{u}$ is the corresponding complex Bloch displace-
 211 ment eigenvector, and $\mathbf{k}$ is the wavevector in the irreducible Brillouin zone (IBZ). The
 212 reduced stiffness and mass matrices $\mathbf{K}_r(\mathbf{k})$ and $\mathbf{M}_r(\mathbf{k})$ are obtained from geometry-
 213 dependent global finite-element matrices $\mathbf{K}$ and $\mathbf{M}$ via a wavevector-dependent Bloch
 214 transformation matrix $\mathbf{T}(\mathbf{k})$,

$$\mathbf{K}_r(\mathbf{k}) = \mathbf{T}(\mathbf{k})^\dagger \mathbf{K} \mathbf{T}(\mathbf{k}), \quad \mathbf{M}_r(\mathbf{k}) = \mathbf{T}(\mathbf{k})^\dagger \mathbf{M} \mathbf{T}(\mathbf{k}), \quad (7)$$

215 with $(\cdot)^\dagger$ the Hermitian transpose. Here $\mathbf{T}(\mathbf{k})$ maps the full mesh degrees of freedom
 216 to an independent set while applying phase factors $e^{i\mathbf{k}\cdot\mathbf{R}}$ across periodic faces of the
 217 unit cell; $\mathbf{K}$ and $\mathbf{M}$ themselves depend only on the material field and are assembled
 218 once per geometry. These intermediate matrices are described briefly so the reader
 219 can compare the explicit FEA pipeline (Figure 3) with the Fourier neural operator
 220 architecture in Section 2.1; they are not inputs to the neural operator.

221 The IBZ is discretized into 25 x wave numbers and 13 y wave numbers, totaling
 222 $25 \times 13 = 325$ unique wavevectors covering the IBZ half plane. For each wavevector,
 223 the six lowest eigenfrequencies (bands) were computed using a custom FEA MATLAB
 224 script (see Section 4 for implementation details).

225 Each eigenvector $\mathbf{u}$ consists of two complex-valued displacement fields, (u_x, u_y) ,
 226 which are separated into their real and imaginary components to produce four
 227 real-valued output channels, $(u_{x,real}, u_{x,imag}, u_{y,real}, u_{y,imag})$. The corresponding eigen-
 228 frequency ω is encoded as a spatially uniform fifth channel, resulting in a target
 229 tensor of size $5 \times 32 \times 32$. Combined with the three-channel material representation

described in the previous subsection, each simulation therefore produces a supervised learning sample mapping

$$\mathbb{R}^{3 \times 32 \times 32} \rightarrow \mathbb{R}^{5 \times 32 \times 32}.$$

The overall finite element data-generation procedure is summarized in Figure 3.

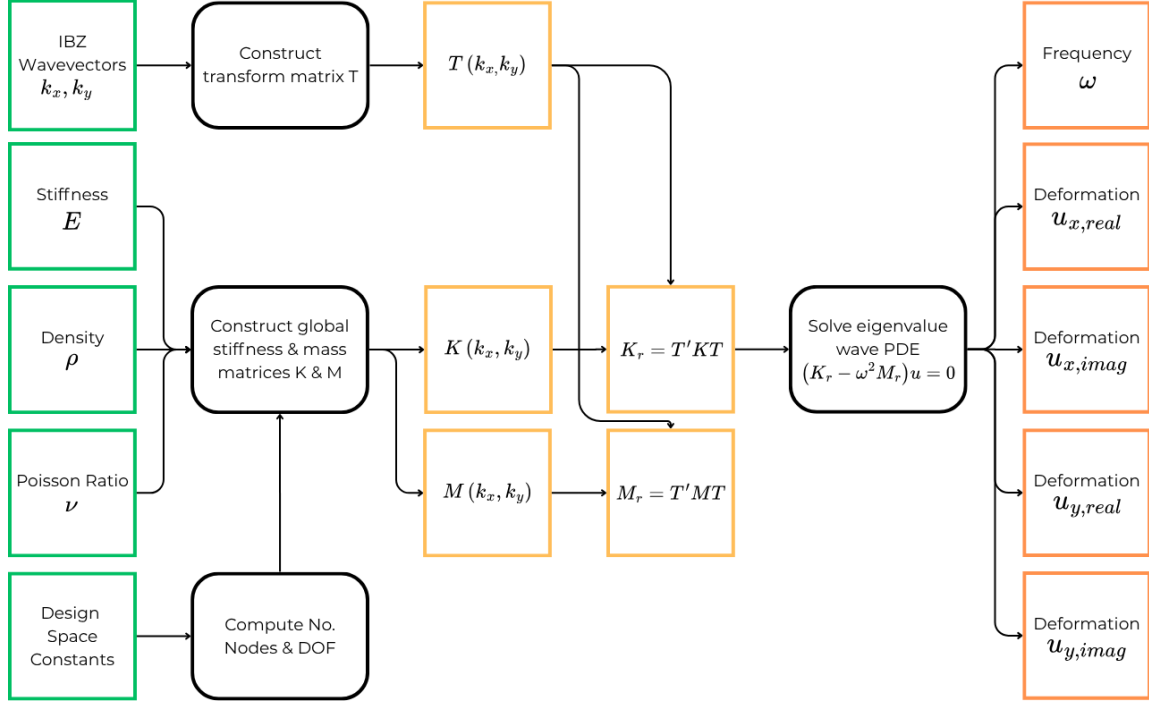


Figure 3: Flowchart of the FEA data generation method. Green squares represent varying inputs to the process, and orange squares represent computed quantities of the FEA process. Black rounded rectangles represent subroutines of the FEA process. The process starts with inputs of defined densities, stiffnesses, and Poisson ratios for each pixel of a metamaterial unit cell geometry (each a 32×32 pixel grid of values ranging from 0 to 1), along with a chosen k_x and k_y wavevector representing the stimulus to the metamaterial. A custom optimized FEA solver takes these inputs and produces intermediate matrices ($\mathbf{K}, \mathbf{M}, \mathbf{T}$), forms the reduced operators ($\mathbf{K}_r, \mathbf{M}_r$), and then solves Eq. (6) for the displacement fields (eigenvectors) and frequencies ω (eigenvalues) for the given inputs.

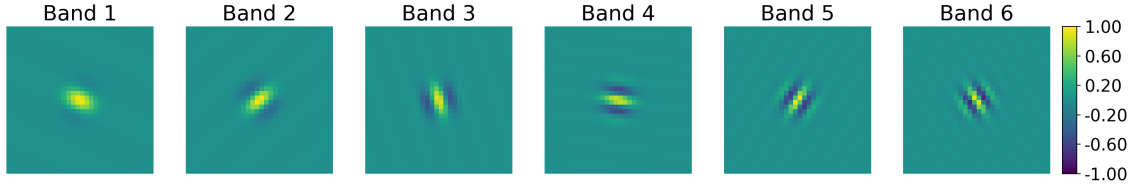
2.4. Input Wavelet Encoding

To allow the FNO model to toggle between PDE solutions for a given geometry, information must be fed to the model indicating which excitation waveform (wavevectors) and deformation mode (band) we are looking for a solution to. This information

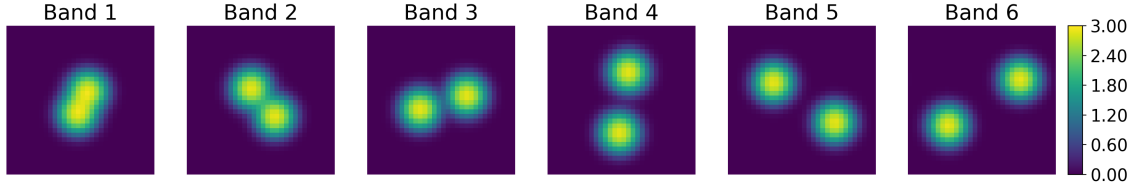
is given in the form of a wavevector embedding, which we have found to yield superior results compared with a constant field embedding (2D matrices of the same value) or a sinusoidal embedding (2D sinusoidal fields with constants encoded as the sinusoidal frequency). A discussion on why wavelet encodings are superior is found in the results presented later in this paper.

For the wavelet encodings, we use a 1D Gabor wavelet transform to map band numbers to unique wavelet images (see Algorithm 1), and a 2D Gabor wavelet transform to map x and y wavevectors to another set of unique wavelet images (see Algorithm 2). These representations show rich variation both in the spatial domain and in the Fourier domain, which allows for FNO model weights of both branches to richly interact with the wavelet representation, leading to better learning outcomes.

add
sec-
tion
refer-
ence
here

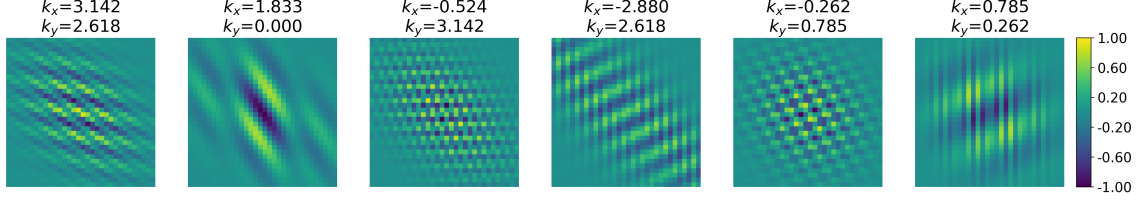


(a) The wavelet spatial encoding for band integer.

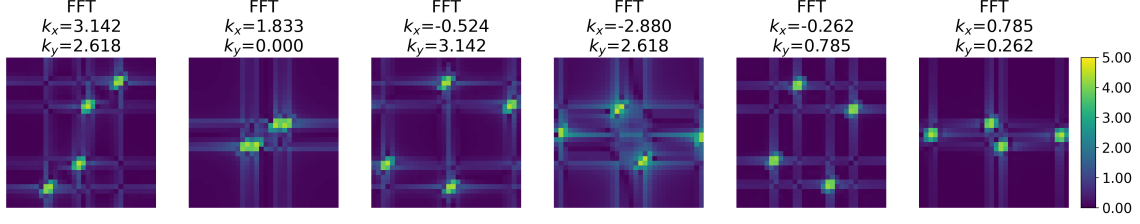


(b) The FFT of the above wavelet spatial encodings.

Figure 4: The wavelet encoding of the band integer (fig. 4a) and its FFT (fig. 4b). Each band corresponds to a unique wavelet with representation in both spatial and spectral domains.



(a) The wavelet spatial encoding for x and y wavevectors.



(b) The FFT of the above wavelet spatial encodings.

Figure 5: The wavelet encoding of the x and y wavevectors (5a) and its FFT 5b. The actual wavevectors are shown in the subplot titles. For each k_x and k_y value pair, there is a unique mapping in both the spatial and spectral domain representations.

2.5. Dataset Construction

Our full dataset contains 24000 geometries, randomly generated as described in prior subsections. Half are discrete valued with only 0s or 1s in each pixel location representing one of the two materials, while the other half are continuous valued with any value between 0 and 1 inclusive at each pixel location, representing blended materials at each pixel location. Each geometry has 6 possible bands each with 325 possible wavevectors (representing a mesh grid the IBZ half plane) per band, resulting in $24000 \times 6 \times 325 = 46,800,000$ sample pairs of inputs with shape $(3 \times 32 \times 32)$ and outputs of shape $(5 \times 32 \times 32)$. The data is stored with float16 datatype format in PyTorch tensors.

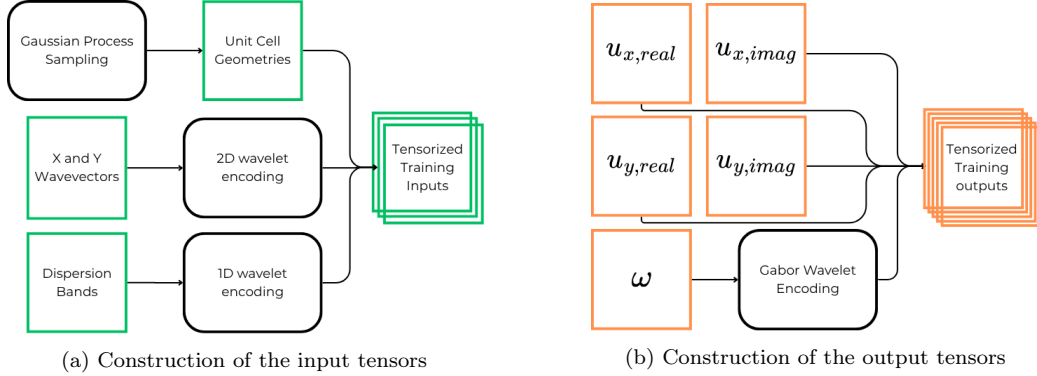


Figure 6: Subfigure a and b shows the process of input data from subsection 2.2 and output data from subsection 2.3 being tensorized for ingestion to the FNO model respectively. Note the wavelet embedding steps on fundamentally scalar quantities.

2.6. Training Procedure

The Fourier Neural Operator was trained using supervised learning on paired inputs and outputs described in Section 2. Each training sample consists of a $3 \times 32 \times 32$ input tensor (geometry and wavelet embeddings) and a $5 \times 32 \times 32$ output tensor (displacement field components and eigenfrequency).

6 loss functions were trialed as performance evaluation criterion: mean absolute error (MAE), mean squared error (MSE), normalized MAE, normalized MSE, Huber (smooth L1), and structural similarity index (SSI). Losses were evaluated and aggregated across all 5 output channels. The best performing loss criterion is the Huber loss and is shown below. Details on the other loss functions trialed can be found in Supplemental Information.

$$\mathcal{L}_{\text{Huber}} = \frac{1}{HW} \sum_{i=0}^{H-1} \sum_{j=0}^{W-1} \sum_{c=0}^4 \begin{cases} \frac{1}{2} (\hat{u}_c(i, j) - u_c(i, j))^2, & |\hat{u}_c(i, j) - u_c(i, j)| \leq \delta, \\ \delta \left(|\hat{u}_c(i, j) - u_c(i, j)| - \frac{\delta}{2} \right), & |\hat{u}_c(i, j) - u_c(i, j)| > \delta. \end{cases}$$

where $H = W = 32$ (pixels) and $c \in \{0, 1, 2, 3, 4\}$ corresponds to the 5 output channels: eigenfrequency, followed by real and imaginary components of x and y displacements.

272 For training the FNO model, combinations of model layers, hidden channels,
273 activation function, loss function, optimizer, and training hyperparameters were
274 trialed. The ADAMW optimizer was found to achieve the best performance compared
275 with other flavors of momentum based optimizers and SGD. The model and training
276 hyperparameters tuned are showed in table S1 in S1, with the best performing
277 combination italicized.

278 The best combination for model hyperparameters was found to be 4 Fourier layers,
279 128 hidden channels, and GELU nonlinear activation function. At this layer count
280 and hidden channel count, performance was asymptotic and greater amounts of either
281 would drive the model further into the overfit region as well as add on additional
282 training time. The best combination for trianing hyperparameters were the Huber loss,
283 which is to use a which would was found to be 12 epochs, Different hyperparameters
284 of FNO model tried in combinations. 4 Fourier layers, with 64 hidden channels and
285 GELU activation function were found to be where performance started to asymptote
286 with increasing model complexity, and so it was not worth the computational costs
287 in training to use more layers or hidden channels. These are the final combination of
288 model hyperparameters from which all data presented in this article is generated.

289 It was found that final model performance did not appreciably change for changes
290 in step size or batch size fairly early on, so those two parameters were left fixed at 4
291 and 256 respectively for the remainder of the training runs. Learning rate and epochs
292 to convergence as expected, had an inverse correlation. The higher the learning rate,
293 the faster convergence happened, but there are two problems with high learning
294 rates. The first is that the model may arrive at a more suboptimal combination of
295 weights if the training rate is too high, because the movements in weight space during
296 each backward pass remain too large to arrive close in weight space to where the
297 loss minima would occur. The second, more interesting phenomena, is that if the
298 learning rate is set higher than $\approx 10^{-2}$, there is instability introduced in the training,
299 whereby at certain epochs, the loss would spike, and never come back down, usually
300 plateauing, in all later epochs. This sort of spike is documented in other applications
301 of ADAM family optimizers indicating the model is pushed into a region of weight
302 space where the loss was flat for movements in any direction.

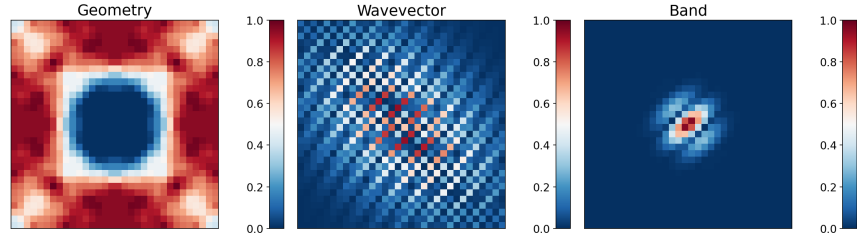
303 3. Results and Discussion

304 This section presents the final model results in the following cases:

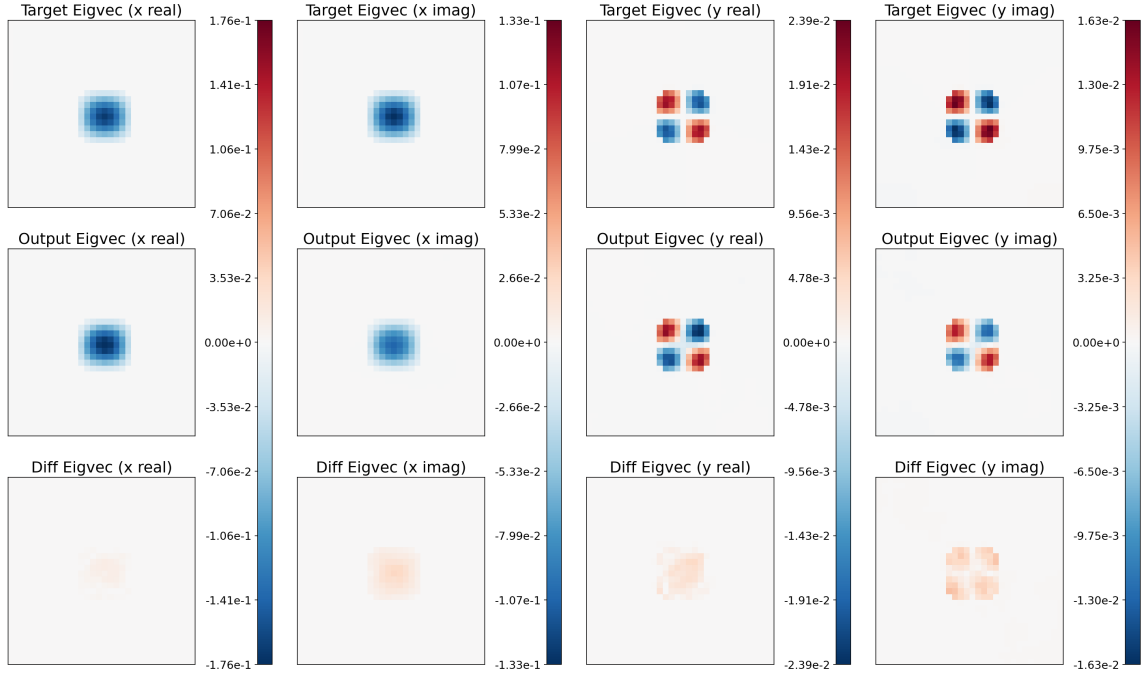
- 305 ✓ Performance on displacement field prediction
- 306 ✓ Performance on eigenvalue prediction
- 307 ✓ Performance across multiple geometries
- 308 ✓ Performance variation with binary geometry discontinuity measures
- 309 ✗ Performance variation with encoding strategies
- 310 ✗ Performance variation with training hyperparameters and loss function

311 3.1. Displacement Field Prediction

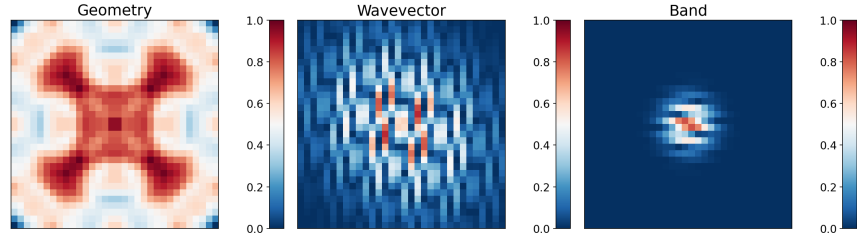
312 We next evaluate the trained Fourier Neural Operator on its predictions of the
313 Bloch displacement fields (u_x, u_y) . For each unseen test sample, the prediction error
314 is measured by the normalized mean absolute error (NMAE) over the four real-valued
315 displacement channels, $(u_{x,\text{real}}, u_{x,\text{imag}}, u_{y,\text{real}}, u_{y,\text{imag}})$. Ranking samples by this NMAE
316 defines a performance distribution from which we select representative cases at the
317 25th, 50th, and 75th percentiles, corresponding to relatively weak, median, and
318 relatively strong predictions. The same percentile comparison is shown for both
319 the binarized and continuous material datasets, allowing direct visual assessment of
320 how well the model recovers the spatial structure of the displacement modes across
321 contrasting geometry regimes.



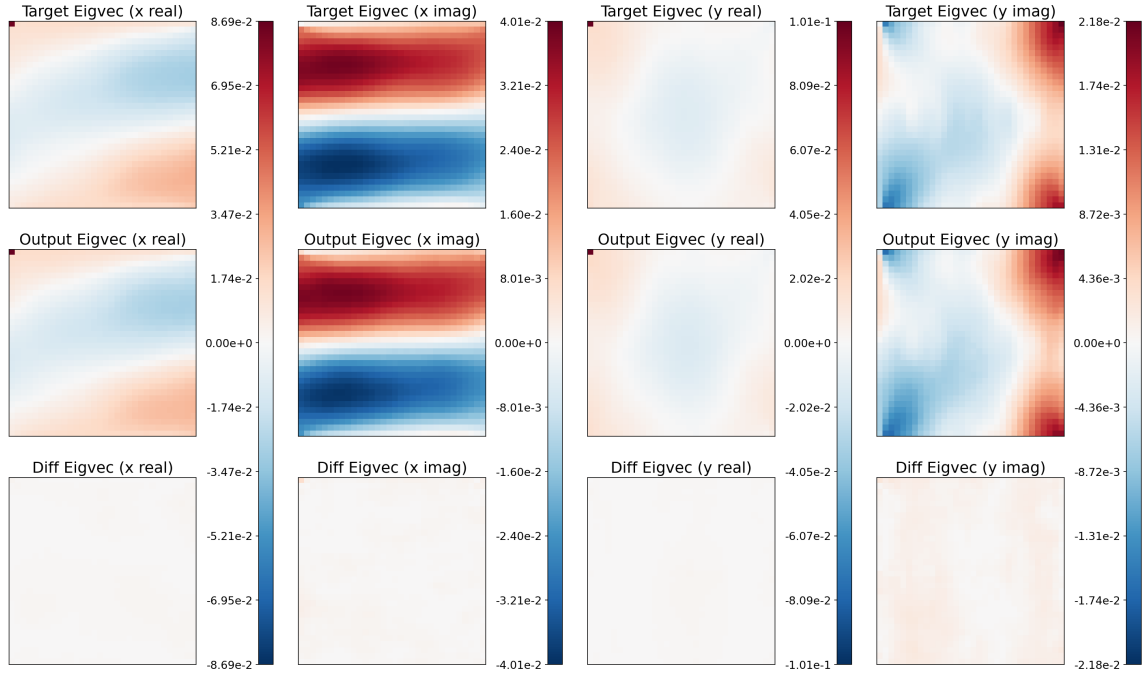
(a) Input sample at the 25th percentile of performance for unseen continuous valued geometries.



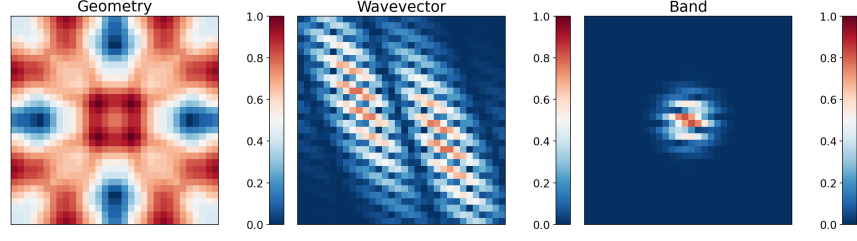
(b) Output predictions at the 25th percentile of performance for unseen continuous valued geometries. The first row are target fields, followed by prediction fields on the second row, and difference fields on the third. At the 25th percentile, prediction outcomes are already fairly good, with accurate representations of scale and structure.



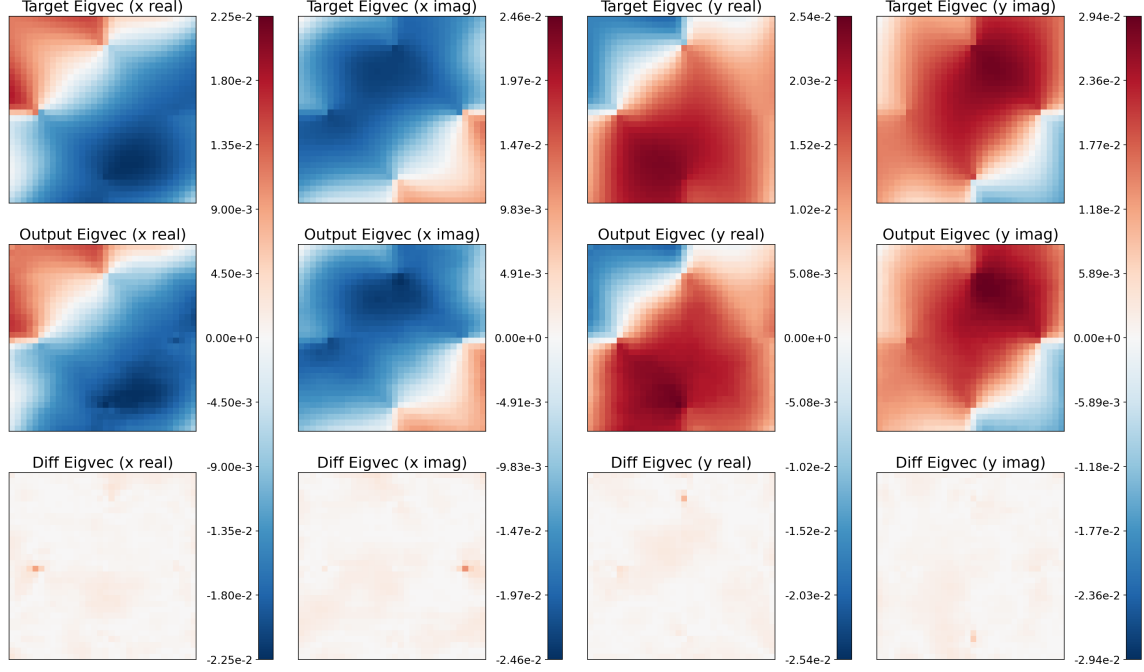
(a) Input sample at the 50th percentile of performance for unseen continuous valued geometries.



(b) Output predictions at the 50th percentile of performance for unseen continuous valued geometries. The first row are target fields, followed by prediction fields on the second row, and difference fields on the third. Predictions are able to consistently capture complex structures without graininess.

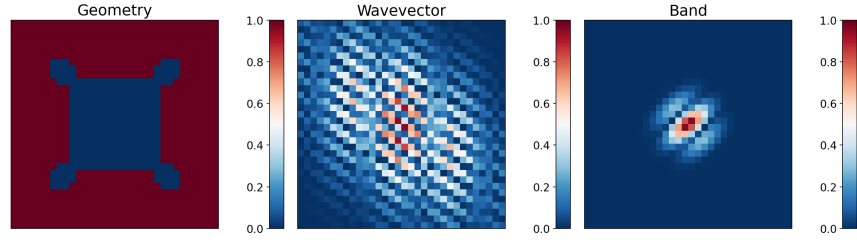


(a) Input sample at the 75th percentile of performance for unseen continuous valued geometries.

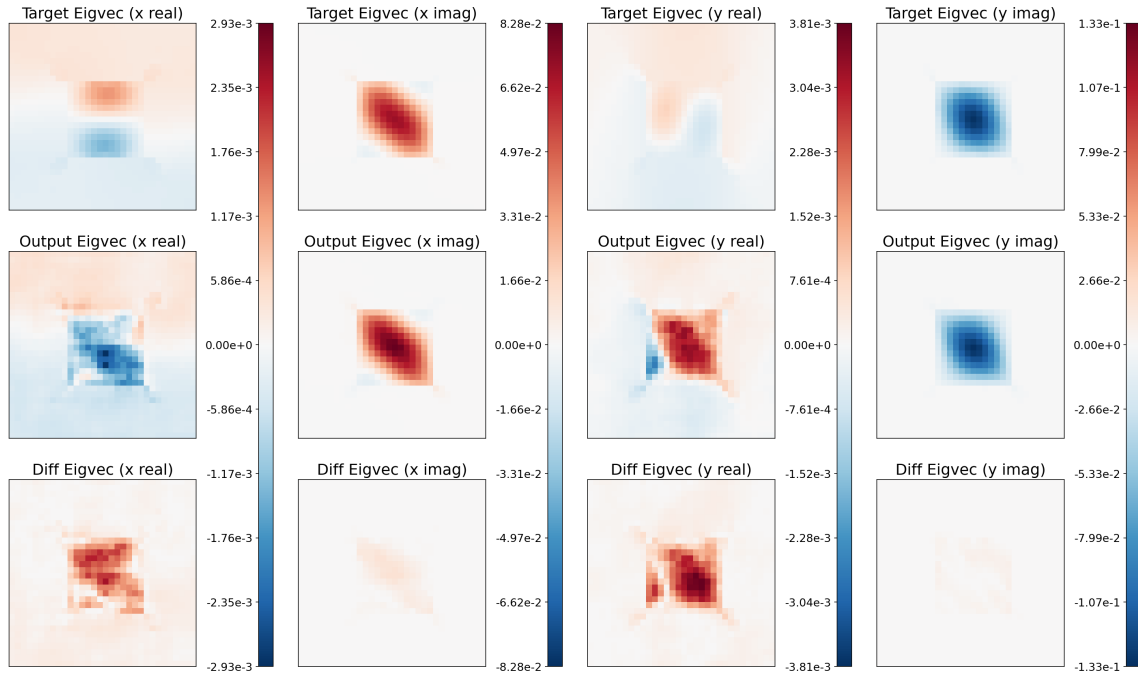


(b) Output predictions at the 75th percentile of performance for unseen continuous valued geometries. The first row are target fields, followed by prediction fields on the second row, and difference fields on the third. At this point, the target and prediction are virtually indistinguishable by eye.

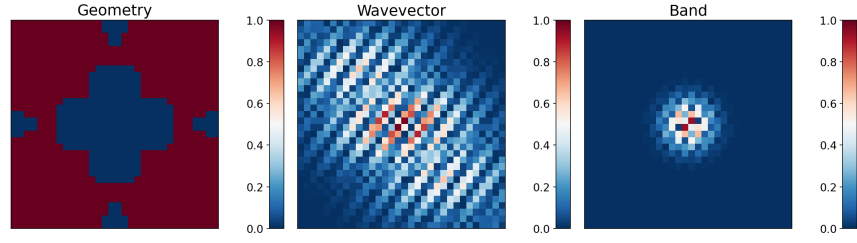
323 These results demonstrate that with wavelet encodings, the FNO model performs
 324 well on continuous geometries, faithfully reproducing the displacement fields in most
 325 cases. Model predictions in the top three quartiles match targets well in structure
 326 and scale, with decreasing average pixel relative errors in the scale of 10^{-2} smoothly
 327 decreasing down to 10^{-4} with increasing percentile performance. (See Figure ?? for
 328 more detailed statistics.)



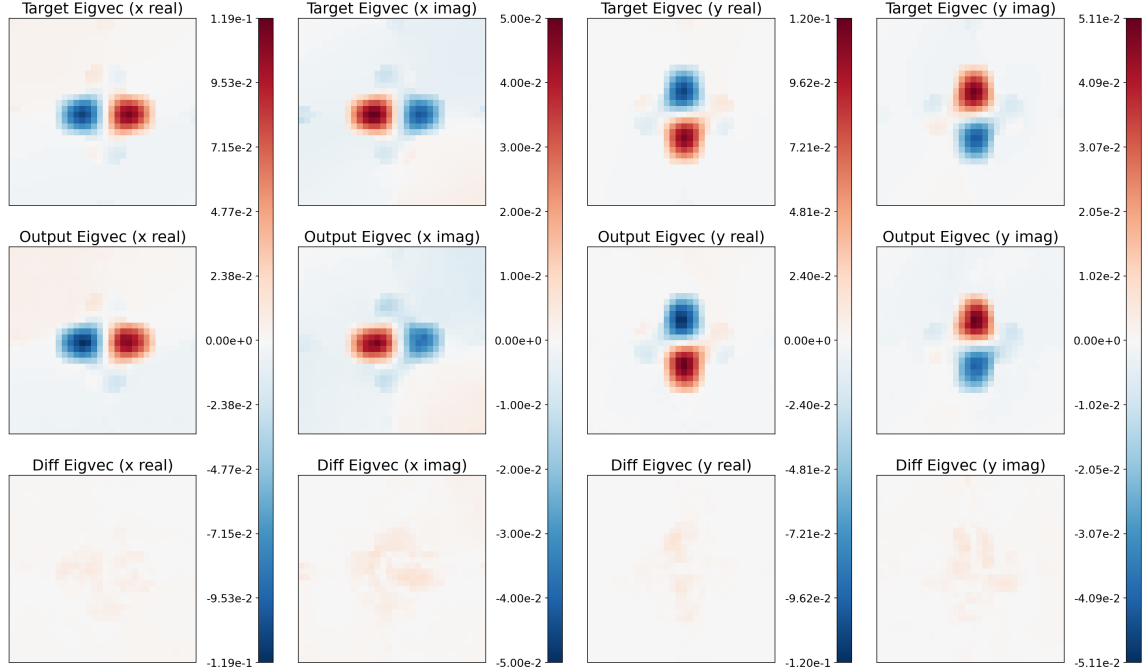
(a) Input sample at the 25th percentile of performance for unseen discrete valued geometries.



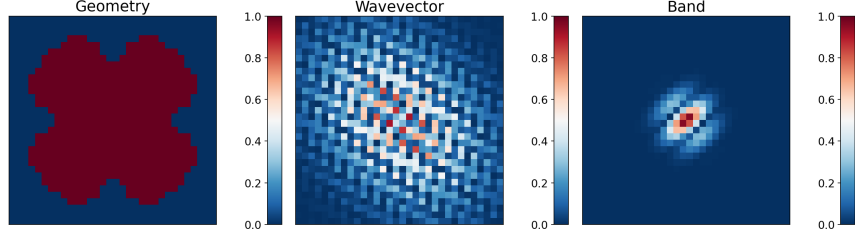
(b) Output predictions at the 25th percentile of performance for unseen discrete valued geometries. The first row are target fields, followed by prediction fields on the second row, and difference fields on the third. Due to the nature of the loss function training, for this particular sample, the model would have prioritized the two imaginary displacement fields, because they are 1 to 2 orders of magnitude larger in scale compared to the two real displacement fields. In other words, absolute error is low at this point, but relative error is not for some channels of small scale than other channels. (Note that the colorbars for each column are independently scaled.)



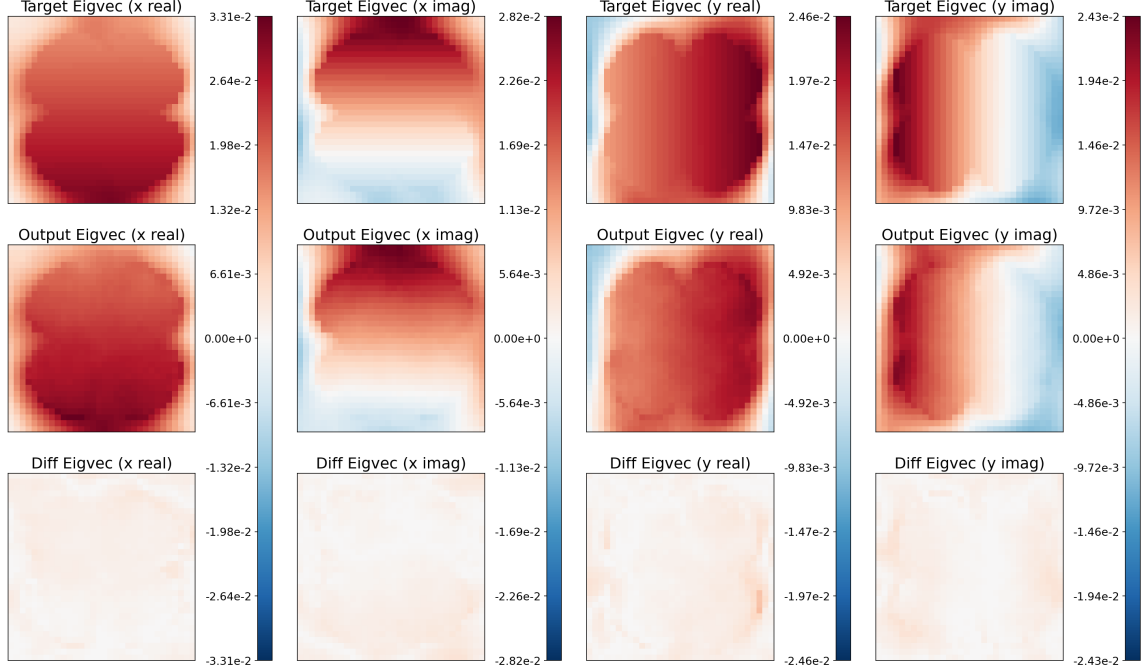
(a) Input sample at the 50th percentile of performance for unseen discrete valued geometries.



(b) Output predictions at the 50th percentile of performance for unseen discrete valued geometries. The first row are target fields, followed by prediction fields on the second row, and difference fields on the third. At this point, absolute and relative errors are low and there is high agreement in structural and scale between targets and predictions.



(a) Input sample at the 75th percentile of performance for unseen discrete valued geometries.



(b) Output predictions at the 75th percentile of performance for unseen discrete valued geometries. The first row are target fields, followed by prediction fields on the second row, and difference fields on the third. At this point, disagreements between targets and predictions become very hard to see by eye.

330 Like with the continuous geometry case, these results demonstrate that with
 331 wavelet encodings, the FNO model performs well on discrete geometries, faithfully
 332 reproducing the displacement fields in most cases. Model predictions in the top three
 333 quartiles match targets well in structure and scale, with decreasing average pixel
 334 relative errors in the scale of 10^{-2} smoothly decreasing down to 10^{-4} with increasing
 335 percentile performance. (See Figure ?? for more detailed statistics.)

3.1.3. Continuous Versus Discrete Performance

The model being able to simultaneously predict well on both continuous and discrete geometries is a non-trivial and important takeaway for the capabilities of FNOs with paired with wavelet encodings. Prior encoding strategies of constant fields and sinusoidal fields delivered far worse results with grainy structures or mode collapse issues because conditioning constants (wavevector and band) information could not propagate well through the spectral branch of the FNO layers. The performance over all unseen samples in the test set are shown in ??.

The poorer performance on binarized geometries is consistent with basic operator-learning theory. Continuous material fields lie in smoother function classes, whereas binary two-phase designs contain jump discontinuities with nonexistent derivatives at material interfaces. Since an FNO approximates the FEA operator $\mathcal{G} : \mathcal{U} \rightarrow \mathcal{V}$ through truncated Fourier representations, it is naturally advantaged when both inputs and outputs are comparatively smooth. Discrete geometries thus perform worse due to sharp boundaries, localized gradients, and broadband high-wavenumber content that are poorly captured on a fixed 32×32 grid and are mismatched to smooth wavelet encodings. Additionally, because continuous geometries have a greater tendency to have large sections of fields with zero or low displacements, a small spike is seen in relative error performance due to the near zero displacements which results in a small denominator.

These results indicate two important takeaways. The first is that it is possible for FNOs with wavelet encodings to learn multiple deformation modes for each geometry which the user can toggle between arbitrarily by setting the wavevector and band inputs. This is to our knowledge the first demonstration of FNOs on solving eigenvalue PDEs. The second, is that it is possible for that same model to simultaneously learn very different distributions of geometries (continuous valued and discrete valued), indicating it might be possible to distill out a generalized understanding of the PDE itself independent of geometry distributions and statistical artifacts. This is a possible subject for future work.

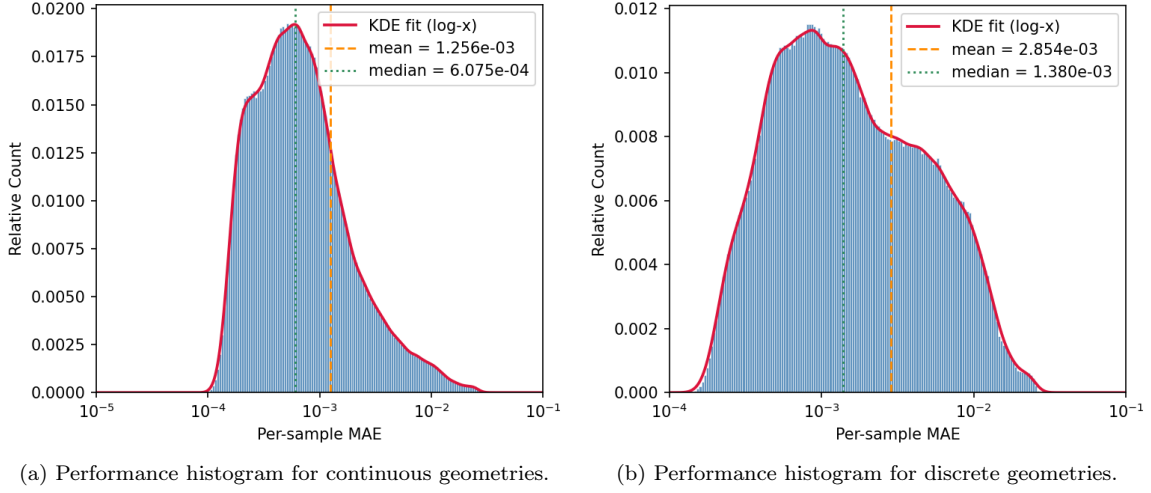


Figure 13: Comparison of relative prediction error distributions for continuous-valued and discrete metamaterial geometries for the same model. While there are some tail outliers, most samples fall within the range of 10^{-2} to 10^{-4} , indicating that the same model is able to learn continuous and discrete geometry cases.

3.2. Dispersion Band Reconstruction

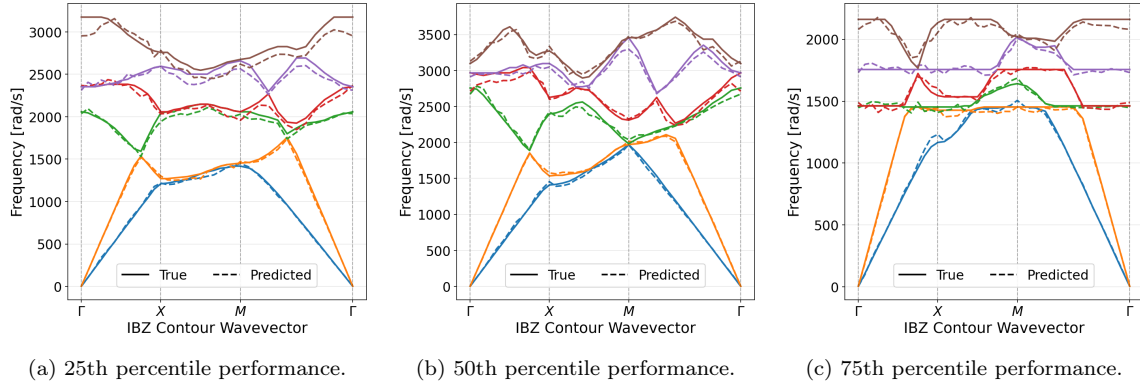


Figure 14: Predicted and ground-truth dispersion bands for representative unseen continuous valued geometries spanning the 25th, 50th, and 75th percentiles of prediction performance.

These plots represent the eigenfrequency predictions of the model across all 325 wavevectors forming the half plane for continuous valued geometries, downselected to be the horizontal, vertical, and diagonal traversal of the IBZ. Note that because the encoding of the eigenfrequency into a field has a log operation (to allow the model

to learn relationships at multiple scales), errors tend to be relative to the scale of the target, and thus higher bands may seem to be more deviant on a linear scale plot. Overall, the model performance is quite good at predictions in the encoded eigenfrequency, showing an average of $< 1\%$ prediction error across all unseen samples.

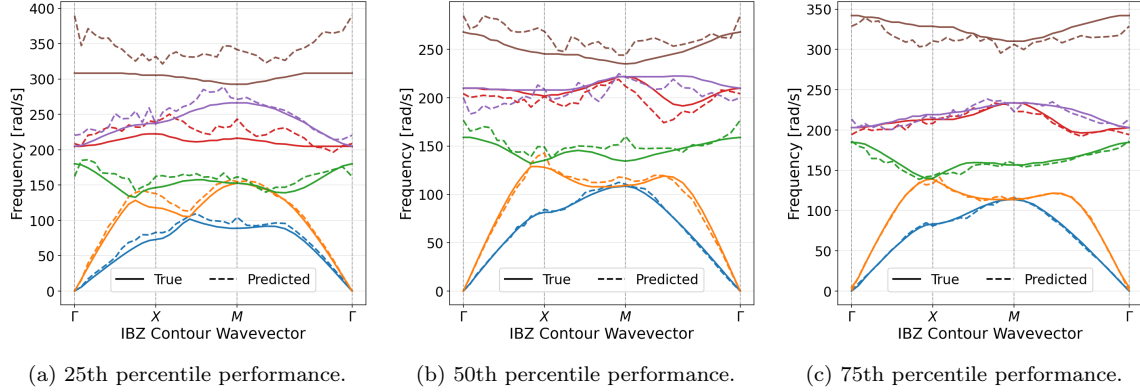


Figure 15: Predicted and ground-truth dispersion bands for representative unseen discrete valued geometries spanning the 25th, 50th, and 75th percentiles of prediction performance.

Like with the continuous geometry figures, these plots represent the eigenfrequency predictions of the model across all 325 wavevectors forming the half plane for discrete valued geometries, downselected to be the horizontal, vertical, and diagonal traversal of the IBZ. Overall, the model performance is quite good at predictions in the encoded eigenfrequency, showing an average of $< 1\%$ prediction error across all unseen samples.

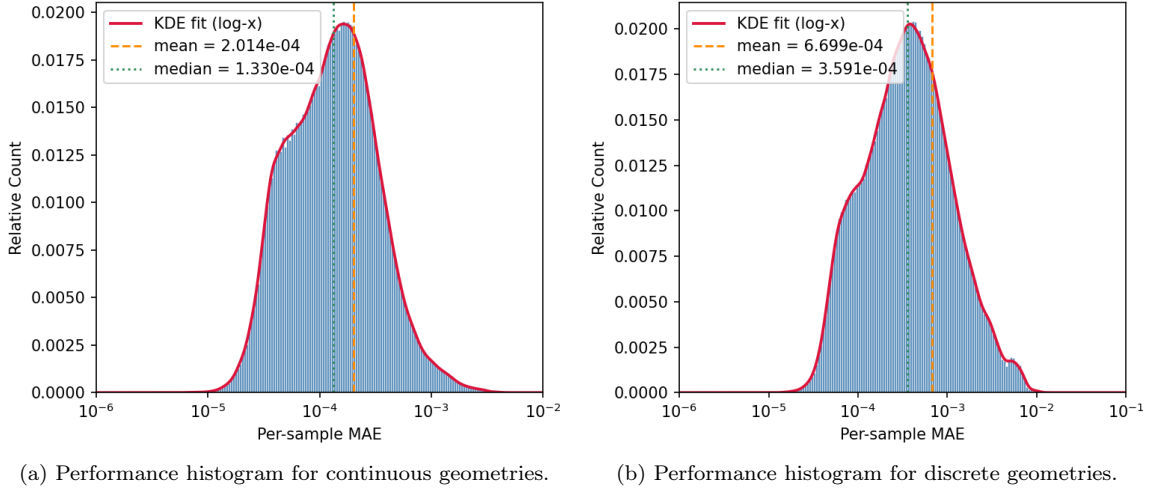


Figure 16: Comparison of relative prediction error distributions for continuous-valued and discrete metamaterial geometries for the same model. While there are some tail outliers, most samples fall within the range of 10^{-2} to 10^{-4} , indicating that the same model is able to learn continuous and discrete geometry cases.

3.3. Dependence of Prediction Error on Band and Boundary Length

The preceding subsections showed that, although the same wavelet-conditioned FNO can predict displacement fields for both continuous and discrete unit cells, discrete geometries are systematically more difficult, with absolute and relative errors larger (Figures 13, 16), and the harder quartiles of the discrete test set degrading more visibly than their continuous counterparts. This gap is expected from the architecture of the FNO itself. Each Fourier layer retains only a truncated set of spectral modes after the discrete FFT. Smooth continuous material fields, and the comparatively smooth displacement modes they induce, concentrate their energy in a modest number of low-to-moderate wavenumbers and are therefore well represented by that truncated Fourier basis. Binary geometries, by contrast, contain jump discontinuities at material interfaces. Under an FFT, such discontinuities produce a broadband spectrum whose coefficients decay slowly with wavenumber, so a substantial fraction of the high-frequency content needed to resolve sharp phase boundaries lies outside the retained modes. The spectral branch therefore sees a degraded, band-limited version of the interface, yielding Gibbs-type ringing or blur and a harder operator-learning problem

precisely when geometric discontinuity is more severe. If this spectral-truncation explanation is correct, then even among discrete geometries the prediction error should increase with the boundary interface length. To test this empirically, we define the *boundary length* of each binarized unit cell as the number of interior four-connected pixel edges separating a 0-valued pixel from a 1-valued pixel. This scalar is a discrete measure of interface complexity. For each of the 1000 unseen discrete test geometries, we compute the mean absolute error (MAE) of the predicted displacement channels, averaged over all 325 IBZ wavevectors, separately for each of the six retained eigenbands. The resulting relationships are shown in Figure 17.

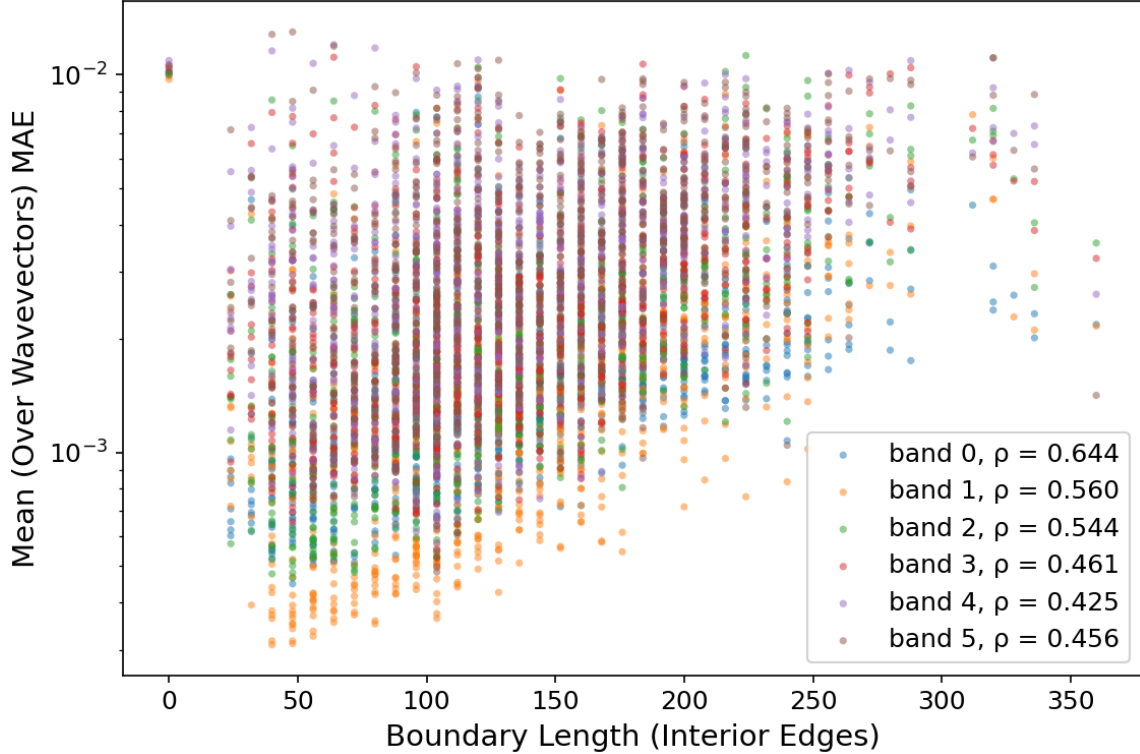


Figure 17: Mean displacement MAE versus interface boundary length, separated by color for each of the six eigenbands on the discrete test set. Points correspond to average performance on unit-cell geometries. Legend values ρ are Spearman rank correlations. A positive trend appears for every band, consistent with degraded FNO accuracy as the total length of 0/1 material interfaces increases.

Two complementary trends are apparent. First, for every band there is a positive

405 monotonic association between boundary length and MAE, with Spearman corre-
 406 lations ranging from $\rho = 0.644$ for band 0 to $\rho \approx 0.42$ – 0.56 for the higher bands.
 407 Geometries with short interfaces are concentrated at lower absolute error, while
 408 geometries with long, meandering phase boundaries systematically occupy the upper
 409 portion of the cloud. This supports the interpretation that the hard cases for the
 410 learned operator are those for which the truncated Fourier representation must resolve
 411 more extensive sharp material transitions on the fixed 32×32 grid. Second, absolute
 412 error itself increases with band index. Averaged over the test geometries, the mean
 413 wavevector-averaged MAE rises from approximately 1.6×10^{-3} for band 0 to approxi-
 414 mately 3.8×10^{-3} for band 5. Higher bands correspond to more rapidly oscillating
 415 Bloch modes that concentrate energy nearer material interfaces and therefore inherit
 416 more of the geometric difficulty encoded by boundary length. Together with the
 417 continuous-versus-discrete comparisons above, Figure 17 indicates that prediction
 418 difficulty on discrete designs is organized both by band (modal complexity) and by
 419 interface measure (geometric complexity), as expected when a spectral surrogate
 420 retains only a portion of the broadband Fourier content generated by discontinuities.

421 *3.4. Band and Wavevector Encoding*

422 In solving this eigenvalue PDE with a FNO model, the first problem to solve
 423 was how to represent the constant parameters fed into the PDE. Specifically, for
 424 our PDE in equation 6, we must specify the wavevectors, as well as which of the
 425 eigenvalue solutions (band) the model needs to solve. (The PDE has multiple
 426 eigenvalues and eigenvectors that satisfy it, corresponding to different deformation
 427 modes at different time frequencies.) Three encoding strategies were tried. The
 428 first is a constant-field representation, which can be considered the naive default
 429 option. In this representation, the two wavevectors k_x , k_y and band b are represented
 430 as $3 \times 32 \times 32$ images with constant pixel values, which would be stacked into a
 431 tensor and fed into the FNO. The problem with this approach is that the FNO model
 432 with this representation tends to experience mode collapse, being unable to learn
 433 anything, and outputting also constant field solutions. The reason for this is that a
 434 Fourier transform of a constant field would result in only one frequency (the constant

frequency of 0) having a non-zero coefficient. Likewise, The resulting matrix from a fast Fourier transform (FFT) which is what is used in FNO, results in a matrix with all 0 values except for the center pixel which corresponds to the 0 frequency. This renders the frequency space branch of the Fourier layers shown in figure 1 effectively unable to contain or propagate information with this representation past the first layer. This in turn causes the Fourier branch to not be able to contribute at all to the model predictive power as it never contributes to any loss and thus cannot be trained. The remaining spatial domain branch of the FNO appears unable to handle the complexity of the problem. This is not surprising as it is essentially a standard CNN of shallow depth, which has not been shown in literature to be able to learn complex eigenvalue PDEs.

With this insight, the next embedding tried was a sinusoidal encoding, which does have unique representations that are not scaled versions of each other in the spectral domain. Here, we used a spatial domain sinusoid with the x and y frequencies a function of b for the band encoding, and a function of k_x and k_y for the wavevector encodings. The resulting spatial domain encoding and its FFT are shown below.

3.5. Output Datatype

The natural representation of the output data is a complex field for both x and y displacements. However, it was found that the models trained on a $2 \times 32 \times 32$ complex64 tensor representation did not perform statistically significantly differently compared with a $4 \times 32 \times 32$ float16 tensor representation across any hyperparameter setting. In both cases, it takes within 20 epochs for training to converge, and final accuracies evaluated on an unseen test set are within 10% relative difference for all combinations of hyperparameters. There is only one major difference between the two cases, which is that the complex64 case consistently takes 1.5 to $2\times$ longer to train, which is a notable disadvantage. If there was a complex32 data format in Python, it might be competitive against float16, but in its absence, even for a naturally complex tensor, we recommend float16 as the training datatype unless higher precision is absolutely necessary.

464 4. Conclusions

465 In this work, we have demonstrated that Fourier Neural Operators, when combined
466 with structured wavelet encodings of PDE parameters, can serve as efficient and
467 accurate surrogates for eigenvalue PDE solvers, allowing for user toggling between
468 the different solutions deterministically and repeatably by choice of PDE parameters
469 in inputs. The specific example is of a non-trivial 2D PDE based on acoustic wave
470 propagation through metamaterial geometries. It is evident from this work that
471 the choice of embedding when using neural networks or neural operators that have
472 specific transforms built into its architecture should be matched to the transforms.
473 In this particular case, the FNO is actually somewhat of a misnomer, because it
474 contains both a spatial and spectral learned kernel interacting with a spatial and
475 spectral representation of the layer input. In this sense, it is more like a Wavelet
476 Neural Operator. Because of this spatial and frequency domain duality and the FNO
477 simultaneously computing and learning in both domains, wavelet embeddings which
478 preserve both spatial and frequency domain information outperform embeddings
479 which only use one of the two domains.

480 The ability for FNOs paired with wavelet encodings to solve eigenvalue PDEs
481 and generalize well past its training geometries to novel geometries, opens up many
482 opportunities in the metamaterial research field directly, but also many other fields of
483 computational mechanics and physics that also have their own eigenvalue PDEs with
484 input parameters that toggle between different solutions. This model, at 2 GB of size
485 with float16 precision, is very lightweight compared to enterprise FEA software and
486 processes inputs on the order of milliseconds, making it a compelling tool for surrogate
487 modeling and/or iterative design for myriad materials and structure engineering
488 problems. Future work to expand on usability could include generalization tests to
489 different resolutions, using physics informed losses to further enhance performance
490 and/or generalizability, and allowing more parameters such as material constants to
491 be used as toggles for solutions, expanding the problem and solution space to more
492 dimensions.

493 **AI Usage Disclosure**
494

495 **Declaration of Competing Interest**

496 The authors declare that they have no known competing financial interests or
497 personal relationships that could have appeared to influence the work reported in
498 this paper.

499 **CRedit authorship contribution statement**

500 **Data availability**

501 The code and data in this article can be found at this GitHub repository: <https://github.com/truotheresy/NO-2D-Metamaterials>.
502

503 The code for FEA computations can be found at this GitHub Repository: <https://github.com/aco8ogren/2D-dispersion.git>.
504

505 **Acknowledgements**

506 **References**

Add
disclo-
sure
for
how
AI
was
used
in this
project

507 S1. Supplementary Information

508 S1.1. Model and Training Hyperparameters

Hyperparameter	Candidate Settings			
Fourier Layers	4	6	8	12
Hidden Channels	32	64	128	256
Activation Function	ReLU	GELU	tanh	Sigmoid
Loss Criterion	L1	L2	Huber	SSIM
Epochs	8	12	16	20
Learning Rate	10^{-4}	10^{-3}	10^{-2}	10^{-1}
Weight Decay	0	10^{-4}	10^{-3}	10^{-2}
Gamma	0	0.01	0.1	0.5
Step Size	0	1	4	10
Batch Size	64	128	256	512
Data Precision	F8 E4M3	F8 E5M2	F16	F32

Table S1: Combinations tried in grid search for optimal model and training hyperparameters.

509 S1.2. Loss Criterion

510 In the exploration of optimal training protocols for the FNO model, other loss
511 functions were trialed as training criterion: mean absolute error (MAE), mean squared
512 error (MSE), normalized MAE, normalized MSE, Huber (smooth L1), and structural
513 similarity index (SSI). For purposes of comparison, the validation criterion was chosen
514 to be MAE and MSE. Broadly speaking, models that performed the best in terms of
515 MSE also performed the best in MAE. The training criterion that yielded the lowest
516 MAE and MSE loss was the Huber loss. All criterion followed a learning rate of
517 2×10^{-3} with a decay factor per epoch of 0.9. The mathematical definitions for each
518 area as follows. Losses were evaluated and aggregated across all 5 output channels.

- 519 • The mean absolute error (MAE) loss is defined as

$$\mathcal{L}_{\text{MAE}} = \frac{1}{HW} \sum_{i=0}^{H-1} \sum_{j=0}^{W-1} \sum_{c=0}^4 |\hat{u}_c(i, j) - u_c(i, j)|$$

Experimentally, models trained exclusively with MAE produced predictions with sharper interfaces and better-preserved boundary features than those trained solely with MSE. This behavior is consistent with the linear penalty of MAE, which does not disproportionately penalize isolated large residuals near discontinuities. Despite the improved qualitative sharpness, MAE training converged to higher validation MAE and MSE than Huber loss, indicating that the improved preservation of fine structural features came at the expense of overall numerical accuracy. This observation also motivated the hybrid training schedule in which optimization began with MAE before transitioning to MSE, combining the edge-preserving characteristics of MAE with the stronger refinement of large residuals provided by MSE.

- The mean squared error (MSE) loss is defined as

$$\mathcal{L}_{\text{MSE}} = \frac{1}{HW} \sum_{i=0}^{H-1} \sum_{j=0}^{W-1} \sum_{c=0}^4 (\hat{u}_c(i, j) - u_c(i, j))^2$$

The quadratic penalty of MSE assigns increasingly large gradients to larger residuals, causing the optimizer to preferentially reduce regions with the greatest numerical error. As a consequence, the loss favors smooth predictions that minimize the overall energy of the residual rather than preserving sharp transitions. Experimentally, models trained exclusively with MSE produced outputs that reproduced the correct magnitude and large-scale structure of the solution, but exhibited diffuse interfaces and blurred boundaries. Fine-scale features were often replaced by low-amplitude, grainy artifacts, consistent with the tendency of MSE to average over uncertain or discontinuous regions in order to minimize the global squared error.

- A hybrid MAE–MSE training schedule was also evaluated, where the model was optimized using MAE for the first half of the training epochs before switching to MSE for the remaining half. This training strategy approximates the behavior of the Huber loss over the course of optimization. During the early stages of

training, prediction errors are typically large, making the linear penalty of MAE advantageous for preserving sharp interfaces while preventing large residuals from dominating the optimization. As training progresses and the average residual decreases, switching to MSE increases the emphasis on refining the remaining prediction errors and improving overall numerical accuracy.

Experimentally, this training schedule consistently outperformed training with either MAE or MSE alone for the same total number of epochs. The resulting predictions retained substantially sharper boundaries than models trained exclusively with MSE, while achieving lower validation MAE and MSE than models trained exclusively with MAE. Among the non-Huber training strategies investigated, the hybrid schedule produced the best overall balance between qualitative feature preservation and quantitative accuracy, converging to within approximately 10% of the validation error achieved by Huber loss. Although less adaptive than Huber, which transitions between MAE- and MSE-like behavior on a per-residual basis, the epoch-wise transition captures much of the same optimization benefit.

- The structural similarity index measure (SSIM) is defined as

$$\mathcal{L}_{\text{SSIM}} = 1 - \frac{1}{5} \sum_{c=0}^4 \frac{(2\mu_{u_c}\mu_{\hat{u}_c} + C_1)(2\sigma_{u_c\hat{u}_c} + C_2)}{(\mu_{u_c}^2 + \mu_{\hat{u}_c}^2 + C_1)(\sigma_{u_c}^2 + \sigma_{\hat{u}_c}^2 + C_2)}, \quad \text{where}$$

$$\mu_{u_c} = \frac{1}{HW} \sum_{i=0}^{H-1} \sum_{j=0}^{W-1} u_c(i, j), \quad \sigma_{u_c\hat{u}_c} = \frac{1}{HW} \sum_{i=0}^{H-1} \sum_{j=0}^{W-1} (u_c(i, j) - \mu_{u_c})(\hat{u}_c(i, j) - \mu_{\hat{u}_c})$$

A significant limitation of SSIM arises when it is applied to signed-valued fields. Unlike its original application to nonnegative image intensities, a perfect sign-reversed field ($\hat{u} = -u$) can achieve nearly the same SSIM score as a perfect reconstruction ($\hat{u} = u$). Consequently, a model trained solely with SSIM may converge to physically incorrect solutions containing polarity inversions while still minimizing the loss. The following derivation demonstrates the mathematical origin of this ambiguity.

571 For the purposes of the following discussion, we consider the common case
 572 where the stabilizing constants satisfy

$$C_1 \ll 2\mu_{u_c}^2, \quad C_2 \ll 2\sigma_{u_c}^2,$$

573 allowing them to be neglected.

574 For a correct reconstruction,

$$u_c^{(+)} = u_c,$$

575 the SSIM score is

$$\text{SSIM}(u_c, u_c^{(+)}) = \frac{(2\mu_{u_c}^2)(2\sigma_{u_c}^2)}{(2\mu_{u_c}^2)(2\sigma_{u_c}^2)} = 1.$$

576 Now consider a sign-reversed reconstruction,

$$u_c^{(-)} = -u_c.$$

577 The mean reverses sign,

$$\mu_{u_c^{(-)}} = -\mu_{u_c},$$

578 the covariance likewise reverses sign,

$$\sigma_{u_c, u_c^{(-)}} = -\sigma_{u_c, u_c},$$

579 while the variance is unchanged,

$$\sigma_{u_c^{(-)}}^2 = \sigma_{u_c}^2.$$

580 Substituting these relationships into the SSIM expression gives

$$\text{SSIM}(u_c, u_c^{(-)}) = \frac{(-2\mu_{u_c}^2)(-2\sigma_{u_c}^2)}{(2\mu_{u_c}^2)(2\sigma_{u_c}^2)} = 1.$$

581 The numerator therefore differs from that of a correct reconstruction by two

factors of -1 , and thus, under these conditions, SSIM assigns essentially the same similarity score to a field and its sign-reversed counterpart as it does to a perfect reconstruction. This ambiguity constitutes a significant limitation when SSIM is applied to signed-valued regression problems where the polarity of the solution is physically meaningful. During training, the network frequently converged to predictions in which localized regions or the entire image were the negative of the correct solution while still achieving a favorable SSIM objective. Although the resulting fields exhibited accurate interface locations, boundary geometry, and overall morphology, the polarity inversions rendered the predictions physically incorrect. Consequently, SSIM should not be used as the sole training criterion for signed-valued fields unless it is supplemented by an additional loss that explicitly penalizes sign reversals.

- The normalized mean absolute error (NMAE) loss is defined as

$$\mathcal{L}_{\text{NMAE}} = \frac{1}{HW} \sum_{i=0}^{H-1} \sum_{j=0}^{W-1} \sum_{c=0}^4 \frac{|\hat{u}_c(i, j) - u_c(i, j)|}{|u_c(i, j)| + \varepsilon}$$

Unlike MAE, NMAE weights each prediction error by the inverse magnitude of the target value, causing the optimization to place greater emphasis on regions where the ground-truth solution is small in scale. Consequently, the loss seeks to minimize relative error rather than absolute error, preventing low-magnitude features from being dominated by high-magnitude regions during training.

Experimentally, NMAE successfully reduced the relative error in low-amplitude portions of the solution. However, this improvement came at the expense of increased overall MAE and MSE compared with training using the standard MAE loss. For the present application, the absolute magnitude of the prediction error was determined to be of greater practical importance than the relative error. As a result, the improved accuracy in low-valued regions did not outweigh the degradation in overall numerical accuracy, making NMAE a less suitable optimization criterion for the objectives of this work. However, for other applications, NMAE can be a valid and superior training criterion than standard

609 MAE.

- 610 • The normalized mean squared error (NMSE) loss is defined as

$$\mathcal{L}_{\text{NMSE}} = \frac{1}{HW} \sum_{i=0}^{H-1} \sum_{j=0}^{W-1} \sum_{c=0}^4 \frac{(\hat{u}_c(i, j) - u_c(i, j))^2}{(u_c(i, j))^2 + \varepsilon}$$

611 Similar to NMAE, NMSE normalizes the prediction error by the magnitude
612 of the target field, causing the optimization to prioritize relative error rather
613 than absolute error. The quadratic penalty retains the characteristic behavior
614 of MSE by assigning greater weight to larger residuals, while the normalization
615 emphasizes regions where the target magnitude is small.

616 Experimentally, NMSE reduced the relative error in low-amplitude regions
617 compared with standard MSE. However, this improvement was accompanied by
618 increased overall validation MAE and MSE, performing worse than conventional
619 MSE optimization. As with NMAE, the objective of this work was to minimize
620 the absolute prediction error rather than the relative error. Consequently, the
621 increased emphasis placed on low-magnitude regions did not translate into
622 improved overall predictive performance, making NMSE less appropriate for
623 the present application.

624 *S1.3. Output Wavelet Encoding*

625 Unlike with the input wavelets encodings which we already know all the possible
626 decoded quantities, model outputs can fall anywhere within a range of valid values.
627 There are two types of output fields. One is a displacement field, which is directly
628 represented as its raw displacement values and requires no encoding. Four of the
629 five output channels contain this type of information (real and imaginary x and y
630 displacements). The fifth channel however, representing eigenfrequency, carries the
631 same issue as wavevector and band of being a scalar valued quantity by nature. For
632 this quantity, we need a way to encode into a wavelet form that the model can train
633 on effectively and decode back to a numerical representation subsequently.

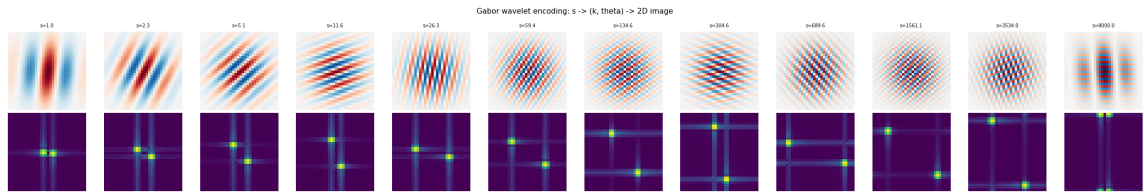


Figure S1: Log scale spaced examples of encoded frequencies from a min of 1Hz to a max of 8000Hz

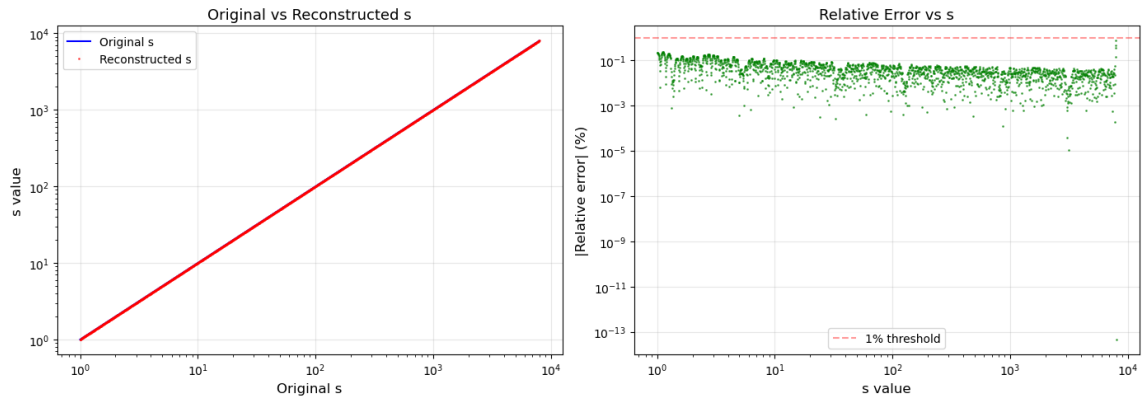


Figure S2: Decoding error of encoded frequencies from a min of 1Hz to a max of 8000Hz

634 S1.4. Algorithms for Gabor Wavelet Encoding

635 This encoding strategy is used for mapping band integers to a 2D wavelet image.

Algorithm 1 1D Gabor Wavelet Embedding from Scalar Input

- 1: **INPUT:** Integer b ; grid size $S \leftarrow 32$; frequency scale $r \leftarrow 2.0$
- 2: Create linspace vectors $x, y \in [-1, 1]$ with S points
- 3: Construct meshgrid $X, Y \leftarrow \text{meshgrid}(x, y)$
- 4: Compute base frequency:

$$f \leftarrow (1.0 + |b|) \cdot \frac{S}{8}$$

- 5: Compute rotation angle:

$$\theta \leftarrow (b \bmod 8) \cdot \frac{S}{16}$$

- 6: Rotate coordinates:

$$X_\theta \leftarrow X \cos \theta + Y \sin \theta, \quad Y_\theta \leftarrow -X \sin \theta + Y \cos \theta$$

- 7: Set Gaussian envelope widths:

$$\sigma_x \leftarrow \sigma_y \leftarrow \frac{0.3}{r}$$

- 8: Compute Gabor wavelet embedding:

$$\psi_b(x, y) \leftarrow \exp \left(-\frac{X_\theta^2}{2\sigma_x^2} - \frac{Y_\theta^2}{2\sigma_y^2} \right) \cdot \cos(f \cdot X_\theta)$$

- 9: **RETURN:** $\psi_b \in \mathbb{R}^{S \times S}$
-

636 In Algorithm 1, a grid size of 32 is used so that the image size matches that of
637 the geometry, for ease of ingestion of the FNO model. The base frequency is set to
638 8, which allows for up to the first 8 bands to be encoded as unique rotations and
639 wavelet spatial frequencies in the domain. The frequency scale r and constants such
640 as 16 and 0.3 were picked so that the images utilized most of the domain and showed
641 sufficient scale variation across the domain.

Algorithm 2 2D Gabor Wavelet Embedding for Wavevectors

- 1: **INPUT:** Integers k_x, k_y ; grid size $S \leftarrow 32$; frequency scale $r \leftarrow 1.0$
- 2: Create linspace vectors $x, y \in [-1, 1]$ with S points
- 3: Construct meshgrid $X, Y \leftarrow \text{meshgrid}(x, y)$
- 4: Compute frequencies:

$$f_x \leftarrow (1.01 + k_x) \cdot \frac{S}{2}, \quad f_y \leftarrow (1.01 + k_y) \cdot \frac{S}{2}$$

- 5: Compute rotation angles:

$$\theta_x \leftarrow (k_x \bmod 5) \cdot \frac{\pi}{5}, \quad \theta_y \leftarrow (k_y \bmod 7) \cdot \frac{\pi}{7}$$

- 6: Rotate coordinates:

$$X_\theta \leftarrow X \cos \theta_x + Y \sin \theta_y, \quad Y_\theta \leftarrow -X \sin \theta_x + Y \cos \theta_y$$

- 7: Set Gaussian envelope widths:

$$\sigma_x \leftarrow \sigma_y \leftarrow \frac{0.4}{r}$$

- 8: Compute Gabor wavelet embedding:

$$\psi_{k_x, k_y}(x, y) \leftarrow \exp\left(-\frac{X_\theta^2}{2\sigma_x^2} - \frac{Y_\theta^2}{2\sigma_y^2}\right) \cdot \sin(f_x X_\theta) \cdot \sin(f_y Y_\theta)$$

- 9: **RETURN:** $\psi_{k_x, k_y} \in \mathbb{R}^{S \times S}$
-

642 Like with Algorithm 1, in Algorithm 2, a grid size of 32 is used for FNO model
643 ease of ingestion. The wavevectors are used to compute the base frequency and
644 rotation angle, with different modulo numbers chosen for x and y to eliminate wavelet
645 image aliasing from x and y being swapped. The frequency scale r of 0.4 was chosen
646 so that the images used most of the domain and showed sufficient scale variation
647 throughout the domain.

648 Unlike with the other quantities, which are only on the input side and always
649 known, this quantity is on the output side, and must be computed, then interpreted.
650 Thus, in addition to an encoding strategy for model ease of learning, we also need
651 a decoding compliment, to transform the model friendly format back to a simple
652 numeric quantity - the frequency response of the material ω at that spatial wave
653 input.

Algorithm 3 Eigenfrequency Wavelet Encoding

- 1: **INPUT:** Eigenfrequency $s > 0$; image size $S \leftarrow 32$; frequency bounds $[s_{\min}, s_{\max}]$; spectral-index bounds $[k_{\min}, k_{\max}]$
- 2: Set constants $\gamma \leftarrow 1$, $\phi \leftarrow 0$, $\sigma \leftarrow 8$, $\theta_{\min} \leftarrow \pi/30$, $\theta_{\max} \leftarrow \pi/2 - \pi/30$
- 3: Compute log-domain values:

$$\ell_s \leftarrow \log s, \quad \ell_{\min} \leftarrow \log s_{\min}, \quad \ell_{\max} \leftarrow \log s_{\max}$$

- 4: Compute normalized position and spectral index:

$$t \leftarrow \text{clip}\left(\frac{\ell_s - \ell_{\min}}{\ell_{\max} - \ell_{\min}}, 0, 1\right), \quad k \leftarrow k_{\min} + t(k_{\max} - k_{\min})$$

- 5: Compute log width per unit k :

$$\Delta_\ell \leftarrow \frac{\ell_{\max} - \ell_{\min}}{k_{\max} - k_{\min}}$$

- 6: Compute orientation from within-band log position:

$$\theta \leftarrow \theta_{\min} + \left(\frac{(\ell_s - \ell_{\min}) \bmod \Delta_\ell}{\Delta_\ell}\right) (\theta_{\max} - \theta_{\min})$$

- 7: Build centered grid $X, Y \in \{-S/2, \dots, S/2 - 1\}$, then rotate:

$$X_\theta \leftarrow X \cos \theta + Y \sin \theta, \quad Y_\theta \leftarrow -X \sin \theta + Y \cos \theta$$

- 8: Set envelope widths $\sigma_x \leftarrow \sigma$, $\sigma_y \leftarrow \gamma \sigma_x$, and carrier frequency $\omega \leftarrow 2\pi k/S$
- 9: Compute wavelet image:

$$\psi_s \leftarrow \exp\left(-\frac{1}{2} \left(\frac{X_\theta^2}{\sigma_x^2} + \frac{Y_\theta^2}{\sigma_y^2}\right)\right) \cos(\omega X_\theta + \phi)$$

- 10: **RETURN:** $\psi_s \in \mathbb{R}^{S \times S}$, k , θ
-

Algorithm 4 Eigenfrequency Wavelet Decoding

- 1: **INPUT:** Wavelet image $\psi \in \mathbb{R}^{S \times S}$; bounds $[s_{\min}, s_{\max}]$, $[k_{\min}, k_{\max}]$, $[\theta_{\min}, \theta_{\max}]$
- 2: Set constants for centroid refinement: radius $R \leftarrow 3$, power $p \leftarrow 2$
- 3: Mean-center image $\psi_c \leftarrow \psi - \text{mean}(\psi)$, then compute the centered 2D discrete Fourier magnitude:

$$\hat{\psi}(k_x, k_y) \leftarrow \sum_{m=0}^{S-1} \sum_{n=0}^{S-1} \psi_c(m, n) \exp\left(-2\pi i \left(\frac{k_x m}{S} + \frac{k_y n}{S}\right)\right), \quad F(k_x, k_y) \leftarrow |\hat{\psi}(k_x, k_y)|$$

- 4: Keep upper-half-plane spectral support (plus positive-frequency center row) and find dominant peak index (r^*, c^*)
- 5: Define symmetric peak partner about center, then compute weighted local centroid around (r^*, c^*) within radius R , using weights $w = F^p$ and excluding points closer to the symmetric partner
- 6: Convert refined centroid to wavevector coordinates (k_x, k_y) , then:

$$k_{\text{ext}} \leftarrow \sqrt{k_x^2 + k_y^2}, \quad \theta_{\text{ext}} \leftarrow \text{atan2}(k_y, k_x) \bmod \pi$$

- 7: Map extracted k to approximate log-frequency:

$$\ell_{\text{approx}} \leftarrow \ell_{\min} + \text{clip}\left(\frac{k_{\text{ext}} - k_{\min}}{k_{\max} - k_{\min}}, 0, 1\right) (\ell_{\max} - \ell_{\min})$$

- 8: Let $\Delta_\ell \leftarrow (\ell_{\max} - \ell_{\min}) / (k_{\max} - k_{\min})$, compute within-band position from angle:

$$\alpha \leftarrow \text{clip}\left(\frac{\theta_{\text{ext}} - \theta_{\min}}{\theta_{\max} - \theta_{\min}}, 0, 1\right), \quad \delta_\ell \leftarrow \alpha \Delta_\ell$$

- 9: Find nearest consistent log-frequency by testing neighboring bands $b-1, b, b+1$:

$$\ell^* \leftarrow \arg \min_{\ell \in \{\ell_{\min} + j\Delta_\ell + \delta_\ell\}} |\ell - \ell_{\text{approx}}|$$

- 10: Clip ℓ^* to $[\ell_{\min}, \ell_{\max}]$, then decode:

$$s_{\text{ext}} \leftarrow \exp(\ell^*)$$

- 11: **RETURN:** $s_{\text{ext}}, k_{\text{ext}}, \theta_{\text{ext}}$
-

654 Algorithms 3 and 4 summarize the forward and inverse mapping used for eigenfre-
655 quency wavelet images. The forward mapping places eigenfrequency on a log scale
656 and jointly encodes magnitude and orientation into a Gabor-like pattern. The inverse
657 mapping recovers an estimate by spectral peak detection with centroid refinement,
658 followed by a consistency step that combines extracted magnitude and angle to
659 reconstruct the most likely log-frequency.